StudyKit

Student: Lyra Kasemi

Project: Individual generative AI application

Purpose

StudyKit helps students turn course notes into an interactive revision session. It generates four key concepts, five flashcards, five multiple-choice questions, and an optional personalized revision plan.

It supports sample database notes, pasted text, and UTF-8 TXT uploads.

Technology and files

- Python and Streamlit provide the application and interface.
- Groq provides API access to the language model.
- The OpenAI Python SDK communicates with Groq through its compatible endpoint: https://api.groq.com/openai/v1
- app.py contains the complete interface, prompts, API integration, validation, quiz scoring, revision feedback, and exports.
- requirements.txt lists the required Python packages.

The Python client is named OpenAI, but requests are sent to Groq.

Installation on Windows

Install Python 3.10 or newer. Download and extract this repository. Open PowerShell in the folder containing app.py, then run:

py -m venv .venv

.\.venv\Scripts\python.exe -m pip install -r requirements.txt

.\.venv\Scripts\python.exe -m streamlit run app.py --server.port 8502

Open http://localhost:8502.

Enter an active Groq API key in the sidebar. Keep it private. Stop the server with Ctrl+C in PowerShell.

Model configuration

The uploaded code defaults to llama-3.3-70b-versatile, which was rejected for this account during development.

Before generating, change the sidebar model to openai/gpt-oss-120b, which was listed in the account's Groq Playground, or another compatible model available to the account.

The current API call uses max_tokens. If a chosen model requires max_completion_tokens instead, update that argument in ask_json() and retest.

Model availability, parameter compatibility, and free-tier quotas depend on Groq's current offerings and the account.

How the application works

1. Accept sample notes, pasted text, or a UTF-8 TXT file.
2. Check input size and length.
3. Send structured instructions and the notes to Groq.
4. Generate four concepts, five flashcards, and five quiz questions.
5. Validate the generated structure and source quotations.
6. Display the study materials and interactive quiz.
7. Calculate the submitted quiz score in Python.
8. Optionally send quiz results and evidence to a second prompt to generate personalized revision advice.
9. Export the session as JSON or a readable Markdown study guide.

The score is calculated against an AI-generated answer key. It does not independently prove that the answer key is correct.

Prompt techniques

Role assignment

The enhanced prompt asks the model to act as a careful, supportive tutor. It requests clear language, coverage of different concepts, and avoidance of trick questions.

Example-based prompting

The Role + few-shot option includes a worked flashcard and quiz item based on a photosynthesis statement. These demonstrate the expected output structure and evidence format.

Zero-shot baseline

The baseline provides the task, output schema, and rules without the tutor role or worked examples. It is available for comparison.

Structured output and source constraints

The prompt requests JSON containing summary, flashcards, and quiz arrays. It instructs the model to use only the supplied notes and include exact source quotations.

Personalized revision prompt

A separate prompt receives the submitted answers, expected answers, calculated correctness, and source excerpts. It generates a revision plan focused on mistakes, or recall practice if all answers are correct.

Design decisions

- Streamlit keeps the interface and processing in Python.
- A single app.py makes the individual project easy to transfer and inspect.
- TXT and pasted notes avoid the complexity of PDF extraction.
- JSON supports consistent rendering and automated validation.
- Source quotations help users trace generated content to their notes.
- Python scoring provides reproducible arithmetic.
- A second API stage bases revision advice on actual submitted answers.
- Session state supports interaction without requiring a database.
- Downloads allow users to preserve results after closing the app.

Adjustable settings

- Model
- Temperature between 0 and 1
- Maximum output tokens between 1000 and 6000

- Baseline zero-shot or Role + few-shot

The session metadata records the model, settings, prompt version, input hash, UTC timestamp, elapsed time, and API-reported token usage. Revision feedback has separate metadata.

Validation and error handling

Input must contain 200–15,000 characters. Uploaded files are limited to 100,000 bytes and must use UTF-8 encoding.

Generated materials are checked for:

- Correct item counts and required text fields.
- Exact quotations occurring in the notes.
- Duplicate flashcard and quiz questions.
- Four distinct options per quiz question.
- Integer answer indices between zero and three.

The app handles missing keys, authentication failures, unsupported requests, connection problems, timeouts, rate limits, server errors, incomplete responses, and invalid JSON.

API requests have a 40-second timeout. Temporary failures are retried up to three times, with one- and two-second waits. Invalid materials are not saved as completed sessions.

Prompt experiments

The Prompt experiments tab describes a manual comparison workflow:

1. Use the same notes and model.
2. Generate with Baseline zero-shot and download the session.
3. Generate with Role + few-shot, keeping other settings unchanged.
4. Compare correctness, clarity, coverage, ambiguity, and source evidence.
5. Compare time and token usage using the saved metadata.
6. Repeat with another temperature and record actual observations.

There is no automated prompt-quality benchmark in this version. A quiz score measures the student's answers against the generated key; it is not a measure of model accuracy.

No completed experimental results are claimed in this documentation. Add actual exports and observations before claiming an improvement.

Verification performed

The uploaded Python file passed syntax checks.

Offline checks verified valid material structure, rejection of fabricated quotations and duplicate options, differences between prompt modes, and readable-guide export.

Streamlit AppTest successfully rendered the initial interface.

These checks did not make live API calls and do not establish successful end-to-end generation or factual accuracy.

Limitations and privacy

AI-generated concepts, questions, answers, and feedback may be wrong. An exact source quotation does not prove that its interpretation is correct. A five-question quiz cannot establish subject mastery.

Remove personal and confidential information before sending notes. Review Groq's current data-use terms.

Keys are entered at runtime and excluded from exports. Downloads contain source excerpts and quiz answers.

Session data may disappear when the session ends. Download results before closing the app or creating a new session.

Alignment with the grading criteria

- Standalone application: Python code with a Streamlit interface.
- API usage: Groq generation and personalized feedback.
- Realistic problem: preparing revision materials from course notes.
- Substantive processing: generation, validation, interaction, scoring, feedback, and export.
- Parameterization: adjustable model and generation settings.
- Prompt techniques: role assignment and worked examples, with a zero-shot baseline.
- Error handling: input checks, response validation, and bounded retries.

- Documentation: explanation of features, techniques, and design decisions.

The supplied rubric makes PDF documentation and GitHub optional for individual work. StudyKit is Lyra Kasemi's individual submission. A grade depends on the professor's evaluation of the working application, understanding, and actual evidence.

AI assistance

Code, troubleshooting, and documentation were developed with AI assistance. The student is responsible for reviewing, testing, understanding, and disclosing that assistance according to course policy.